

Lecture 6 Lab: Simulating Stationarity, MA(1), AR(1), and Differencing

Instructor notebook

September 2026

Contents

Lab roadmap	2
What we are doing today	2
Submission reminder	2
What the five exercises practise	2
0. R tools used throughout the lab	2
Reproducible random simulations: <code>set.seed()</code>	2
White noise: <code>rnorm()</code>	2
ACF and PACF: <code>acf()</code> and <code>pacf()</code>	3
Formula review: γ and ρ for MA(1) and AR(1)	4
Two symbols that must not be confused	4
MA(1) reference	5
AR(1) reference	5
Calculate the reference values in R	5
Plot the two theoretical ACF shapes	6
1. Exercise 1: MA(1) with a constant	7
Generate the MA(1) series: <code>arma.sim()</code>	7
Plot the series and its sample ACF	8
Calculate the reference values in R	10
Optional visual check: theoretical versus sample ACF	11
2. Exercise 2: AR(1)	11
Compare the ACF and PACF	12
Optional numerical comparison of theory and sample values	15
3. Exercise 3: deterministic linear trend plus stationary noise	15
Remove the trend with <code>diff()</code>	16
ACF/PACF before and after differencing	16
4. Exercise 4: random walk with drift	18
5. Exercise 5: compare MA(1) and AR(1) lag-1 autocorrelation	22
Why an MA(1) cannot reach 0.6	22
6. A compact R checklist before submission	23

Lab roadmap

What we are doing today

This is a **code-centered** lab for the five practice exercises at the end of Lecture 6. We will use R to simulate each process, make time plots, and read sample ACF/PACF plots. We will use only the minimum theory needed to understand what the code generates and what each graph means.

The important habit for this lab is:

Write the model in R, make the plot, then explain whether the observed ACF/PACF pattern agrees with the model's expected behavior.

Submission reminder

Submit **both your report and your R code by midnight on Friday, September 24, 2026.**

Your report should not consist only of code and figures. For every exercise, include:

1. the model you simulated or transformed;
2. the key R code, with a short explanation of its purpose;
3. the relevant plot(s) or numerical output; and
4. two or three sentences explaining what you learned from the output.

What the five exercises practise

Exercise	Main R idea	Main plot-reading idea
1	Simulate an MA(1) with <code>arima.sim()</code>	MA(1) sample ACF has one important lag-1 spike; later lags fluctuate around zero.
2	Simulate an AR(1) and compare <code>acf()</code> with <code>pacf()</code>	AR(1) ACF decays; the PACF identifies order 1.
3	Add a deterministic trend, then use <code>diff()</code>	Differencing removes a linear trend.
4	Use <code>cumsum()</code> to create a random walk with drift	A random walk is nonstationary; its first difference is white noise around the drift.
5	Calculate and compare two lag-1 correlations	An MA(1) lag-1 ACF cannot exceed 0.5 in absolute value.

0. R tools used throughout the lab

Reproducible random simulations: `set.seed()`

All examples below use random numbers. `set.seed()` fixes the random-number starting point. This means students who run the same notebook see the same simulated data and the same ACF/PACF plots. It does **not** make the data non-random; it makes the random demonstration reproducible.

```
# Use one fixed seed for the whole classroom notebook.  
set.seed(6)
```

White noise: `rnorm()`

`rnorm(n, mean, sd)` generates `n` independent normally distributed random values. We use it to create white-noise innovations such as w_t or ϵ_t .

```

# Create 400 white-noise innovations with mean 0 and standard deviation 1.
# The object is a numeric vector, not yet a time-series model.
n <- 400
sigma <- 1
white_noise <- rnorm(n = n, mean = 0, sd = sigma)

# Look at only the first few values. The values change from one time point to another
# because they are random innovations.
head(white_noise)

```

```

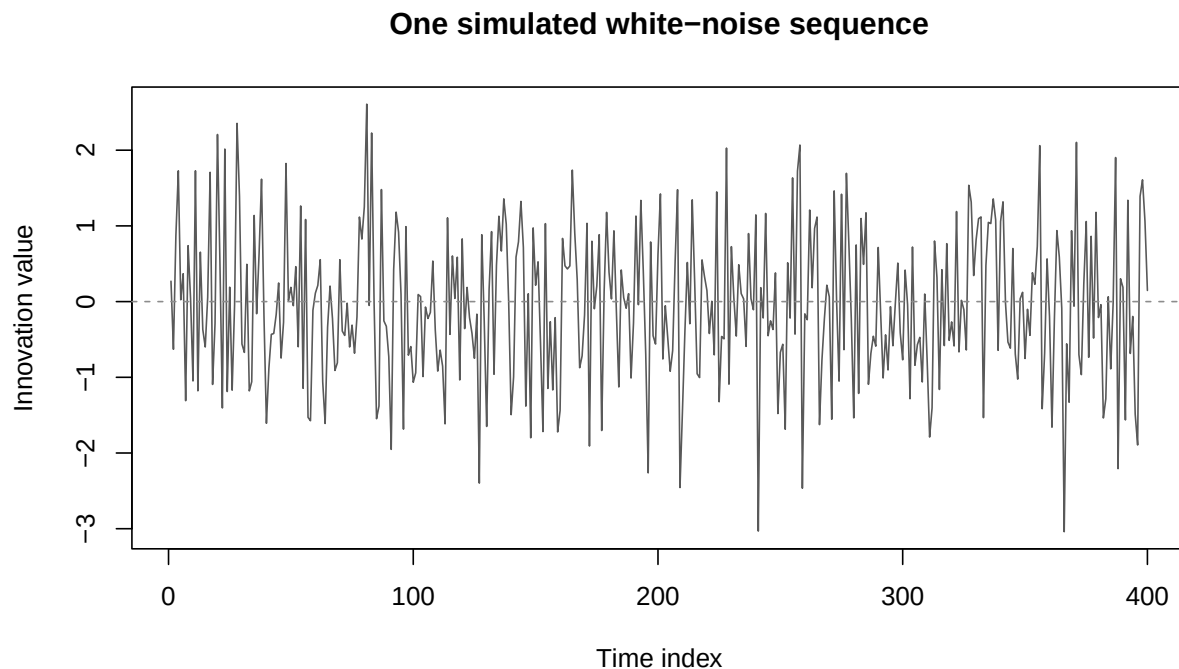
[1] 0.26960598 -0.62998541 0.86865983 1.72719552 0.02418764 0.36802518

```

```

plot(
  white_noise,
  type = "l",
  col = "grey35",
  xlab = "Time index",
  ylab = "Innovation value",
  main = "One simulated white-noise sequence"
)
abline(h = 0, lty = 2, col = "grey55")

```



When reading this plot, do not expect a perfectly flat line. White noise jumps up and down randomly. The useful feature is that it does not show a persistent trend or a repeating pattern.

ACF and PACF: `acf()` and `pacf()`

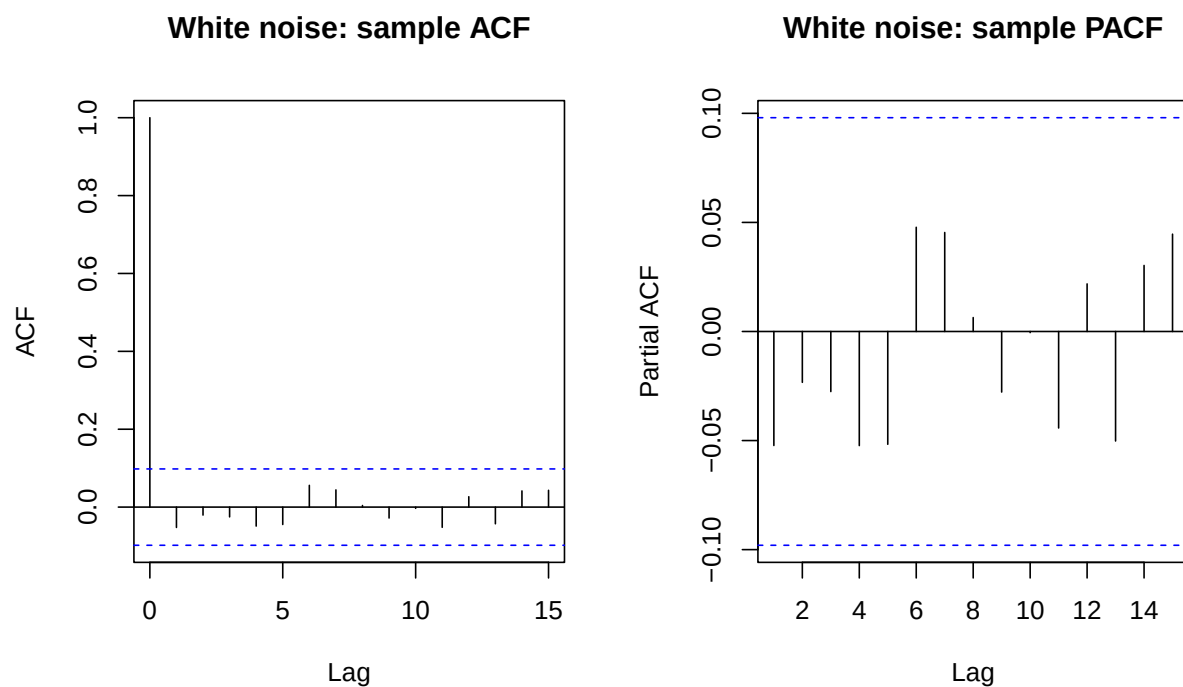
- `acf(x, lag.max = ...)` estimates and plots the **sample autocorrelation** of `x` at several lags.
- `pacf(x, lag.max = ...)` estimates and plots the **sample partial autocorrelation**.
- `lag.max = 15` limits the display to lags 1 through 15. This keeps the figure focused on the short-lag behavior relevant to this lecture.

The dashed horizontal bands on the default plots are rough sampling-noise reference bands. For a finite simulated series, a theoretical correlation of zero will usually appear as a small positive or negative sample bar. Therefore, do not expect later MA lags to be exactly zero in the plot.

```
par(mfrow = c(1, 2))

acf(
  white_noise,
  lag.max = 15,
  main = "White noise: sample ACF"
)

pacf(
  white_noise,
  lag.max = 15,
  main = "White noise: sample PACF"
)
```



```
par(mfrow = c(1, 1))
```

For white noise, most bars should remain inside the reference bands. A few small bars outside a band can occur by chance, especially when we inspect many lags.

Formula review: γ and ρ for MA(1) and AR(1)

Before simulating the five exercises, we need a short formula reference. This section is for **reading the code and interpreting its output**, not for reproducing every probability derivation.

Two symbols that must not be confused

- γ_k , read “gamma at lag k ,” is the **autocovariance** at lag k . In particular, γ_0 is the variance.

- ρ_k , read “rho at lag k,” is the **autocorrelation** at lag k : $\rho_k = \gamma_k/\gamma_0$. It has no units and always lies between -1 and 1.

Therefore, a value such as `gamma0 = 2.7778` is not an ACF bar. It is a variance. A value such as `rho1 = 0.8` is the theoretical height of the lag-1 ACF bar.

MA(1) reference

For the zero-mean MA(1) process

$$X_t = \epsilon_t + \theta\epsilon_{t-1}, \quad \text{Var}(\epsilon_t) = \sigma^2,$$

the useful results are:

$$E(X_t) = 0, \quad \gamma_0 = (1 + \theta^2)\sigma^2,$$

$$\gamma_1 = \theta\sigma^2, \quad \gamma_k = 0 \text{ for } k \geq 2,$$

$$\rho_0 = 1, \quad \rho_1 = \frac{\theta}{1 + \theta^2}, \quad \rho_k = 0 \text{ for } k \geq 2.$$

An added constant changes only the mean. For example, $Z_t = 10 + X_t$ has mean 10 but exactly the same γ_k and ρ_k values as X_t . An MA(1) is stationary for every finite theta; its theoretical ACF cuts off after lag 1.

AR(1) reference

For the zero-mean AR(1) process

$$Y_t = \alpha Y_{t-1} + w_t, \quad \text{Var}(w_t) = \sigma^2,$$

the process is stationary only when $|\alpha| < 1$. Under that condition:

$$E(Y_t) = 0, \quad \gamma_0 = \frac{\sigma^2}{1 - \alpha^2},$$

$$\gamma_k = \alpha^k \gamma_0, \quad \rho_k = \alpha^k \quad (k \geq 0).$$

Thus an AR(1) theoretical ACF does **not** cut off: it decreases geometrically. A positive alpha creates positive decay; a negative alpha makes the ACF alternate signs. The PACF, rather than the ACF, cuts off after lag 1.

Calculate the reference values in R

```
# These objects are examples for reading the formulas. They do not fit a model
# and do not create random data. sigma is 1 because it was set in Section 0.
theta_formula <- -0.6
alpha_formula <- 0.8

# MA(1): gamma0 is variance; gamma1 is lag-1 covariance; rho1 is lag-1 ACF.
ma1_formula_table <- data.frame(
  process = "MA(1)", theta = -0.6,
```

```

gamma0 = (1 + theta_formula^2) * sigma^2,
gamma1 = theta_formula * sigma^2,
rho1 = theta_formula / (1 + theta_formula^2),
rho2_and_beyond = 0
)

# AR(1): 1:5 is c(1, 2, 3, 4, 5). The ^ operator calculates alpha^k at
# each lag, producing the theoretical ACF values rho_1 through rho_5.
ar1_formula_table <- data.frame(
  process = "AR(1)", alpha = 0.8",
  lag = 1:5,
  gamma0 = sigma^2 / (1 - alpha_formula^2),
  rho1_to_rho5 = alpha_formula^(1:5)
)

ma1_formula_table

```

	process	gamma0	gamma1	rho1	rho2_and_beyond
1	MA(1), theta = -0.6	1.36	-0.6	-0.4411765	0

	process	lag	gamma0	rho1_to_rho5
1	AR(1), alpha = 0.8	1	2.777778	0.80000
2	AR(1), alpha = 0.8	2	2.777778	0.64000
3	AR(1), alpha = 0.8	3	2.777778	0.51200
4	AR(1), alpha = 0.8	4	2.777778	0.40960
5	AR(1), alpha = 0.8	5	2.777778	0.32768

Read the two outputs differently:

- `ma1_formula_table` has one row because an MA(1) has only one nonzero positive-lag ACF value, `rho1`. The entry `rho2_and_beyond = 0` summarizes all later theoretical lags.
- `ar1_formula_table` has five rows because `alpha_formula^(1:5)` creates five values. The `lag` column tells us which row is ρ_1 , ρ_2 , and so on. The repeated `process` and `gamma0` entries are intentional: every row refers to the same AR(1), while the last column moves from ρ_1 to ρ_5 .

Plot the two theoretical ACF shapes

```

lags_formula <- 0:15
ma1_rho_formula <- c(1, theta_formula / (1 + theta_formula^2),
  rep(0, length(lags_formula) - 2))
ar1_rho_formula <- alpha_formula^lags_formula

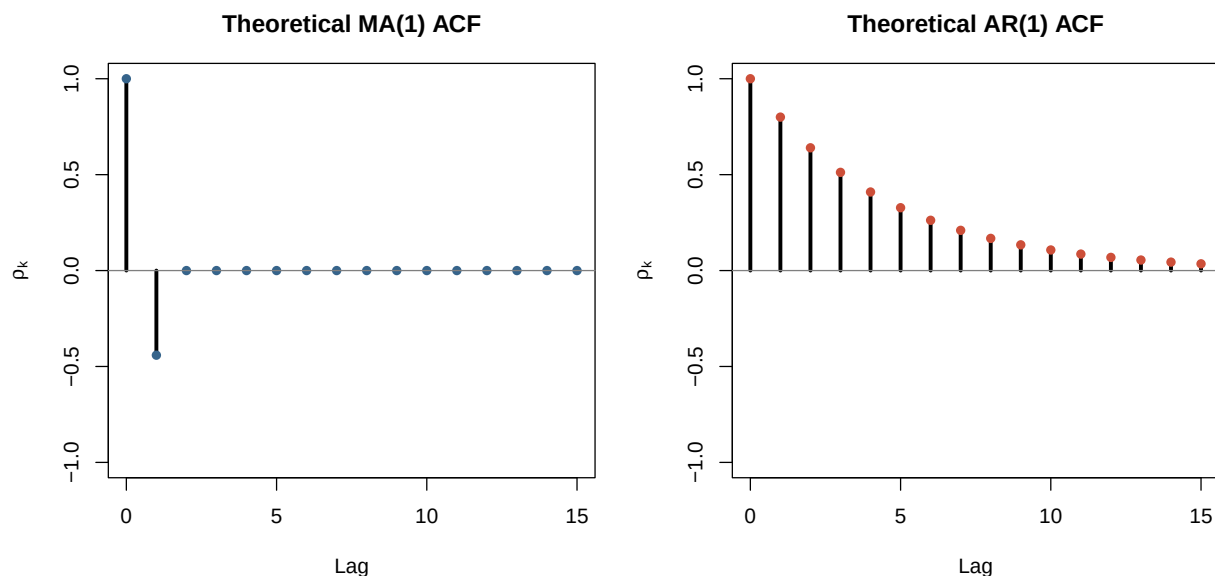
par(mfrow = c(1, 2), mar = c(4.2, 4.5, 3.2, 1))

plot(lags_formula, ma1_rho_formula, type = "h", lwd = 3,
  ylim = c(-1, 1), xlab = "Lag", ylab = expression(rho[k]),
  main = "Theoretical MA(1) ACF")
points(lags_formula, ma1_rho_formula, pch = 16, col = "steelblue4")
abline(h = 0, col = "grey50")

plot(lags_formula, ar1_rho_formula, type = "h", lwd = 3,
  ylim = c(-1, 1), xlab = "Lag", ylab = expression(rho[k]),
  main = "Theoretical AR(1) ACF")

```

```
points(lags_formula, ar1_rho_formula, pch = 16, col = "tomato3")
abline(h = 0, col = "grey50")
```



```
par(mfrow = c(1, 1), mar = c(5.1, 4.1, 4.1, 2.1))
```

The left figure is a **cutoff**: only lags 0 and 1 are nonzero. The right figure is a **tail-off**: every lag remains positive, but the bars gradually shrink. These are theoretical plots, so they contain no blue sampling bands and no random fluctuations. The sample ACF plots later in the notebook will only approximate these clean shapes.

1. Exercise 1: MA(1) with a constant

The exercise model is

$$Z_t = 10 + \epsilon_t - 0.6\epsilon_{t-1}.$$

The constant 10 sets the series' average level. The MA part describes how the current observation combines the current shock with the previous shock.

Generate the MA(1) series: `arima.sim()`

`arima.sim()` generates a simulated ARMA time series. For the MA(1) call below, read its inputs from the inside out:

- `model = list(ma = -0.6)` is a **named list** describing the model. The name `ma` tells R to use a moving-average term; `-0.6` is its one coefficient, so this is MA order 1. We do not write `ma = "-0.6"`, because the coefficient must be a number, not text.
- `n = 400` requests 400 output observations. A longer series makes the sample ACF less noisy than a very short series would be.
- `innov = innov_ma1` supplies the actual white-noise innovation vector ϵ_t used by the simulation. `innov` is short for **innovations**: the new random shocks entering the model at each time point.

If we omitted `innov`, `arima.sim()` could generate its own innovations. We create and name them ourselves because then students can see exactly where the ϵ_t 's in the mathematical model enter the R call. `arima.sim()`

returns a one-dimensional `ts` object containing the zero-mean MA part. We add 10 afterward so that the simulated series matches Z_t , whose model mean is 10.

```
# Keep the innovations as a separate object. It is the sequence epsilon_t in  
# the model, with one fresh random shock at each simulated time point.  
innov_ma1 <- rnorm(n = n, mean = 0, sd = sigma)  
  
# arima.sim() output is the zero-mean random MA(1) component  
# epsilon_t - 0.6 * epsilon_(t-1). The named list says this is an MA model  
# with one coefficient -0.6; n controls the output length; innov supplies  
# the white-noise shocks created immediately above.  
ma1_random_part <- arima.sim(  
  model = list(ma = -0.6),  
  n = n,  
  innov = innov_ma1  
)  
  
# Add the constant level from Z_t = 10 + epsilon_t - 0.6 epsilon_(t-1).  
z_ma1 <- 10 + ma1_random_part  
  
# A ts object prints with a time index, but it also behaves like a numeric series.  
class(z_ma1)
```

```
[1] "ts"
```

```
length(z_ma1)
```

```
[1] 400
```

```
head(z_ma1)
```

```
[1] 8.789929 9.612748 10.658218 9.040751 9.823717 9.991176
```

```
mean(z_ma1)
```

```
[1] 9.980427
```

Read this output carefully:

- `class(z_ma1)` should include `"ts"`. This tells us that the result is stored as a time series, so time-series plotting functions can use it naturally.
- `length(z_ma1)` should be 400. Adding the constant 10 does not add extra observations; it shifts each of the same 400 values upward.
- `head(z_ma1)` shows only the first six simulated observations, not the model's six parameters.
- `mean(z_ma1)` is the **sample mean** from this one simulation. It should be near 10, but it is not required to equal 10 exactly. The model mean is 10; the simulated sample has random variation.

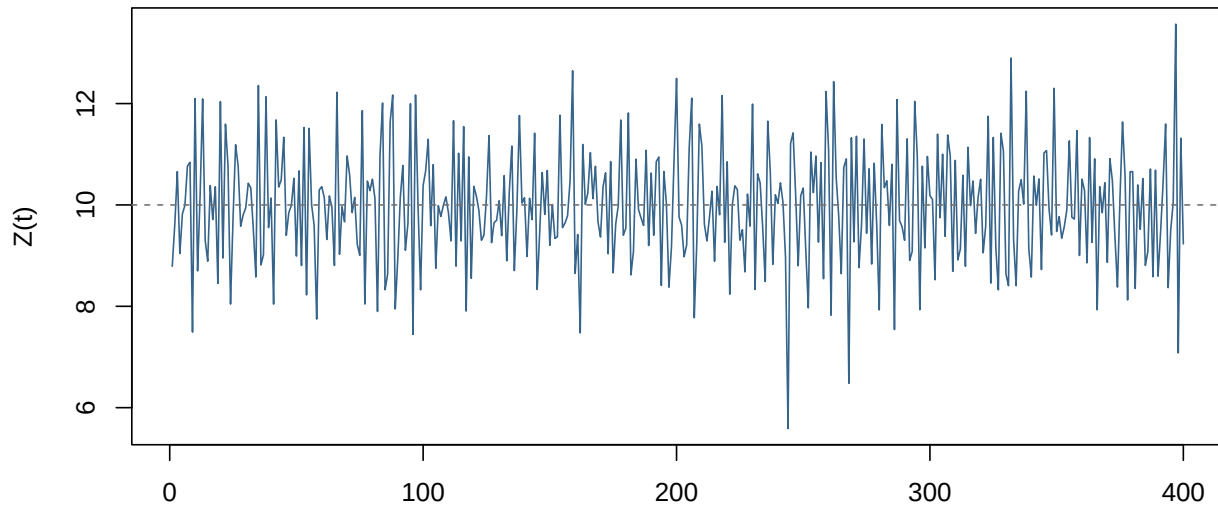
Plot the series and its sample ACF

```
par(mfrow = c(2, 1), mar = c(3.5, 4, 2.5, 1))  
  
plot(  
  z_ma1,  
  type = "l",  
  col = "steelblue4",  
  xlab = "Time index",  
  ylab = "Z(t)",  
  main = "Exercise 1: simulated MA(1) series around mean 10"
```

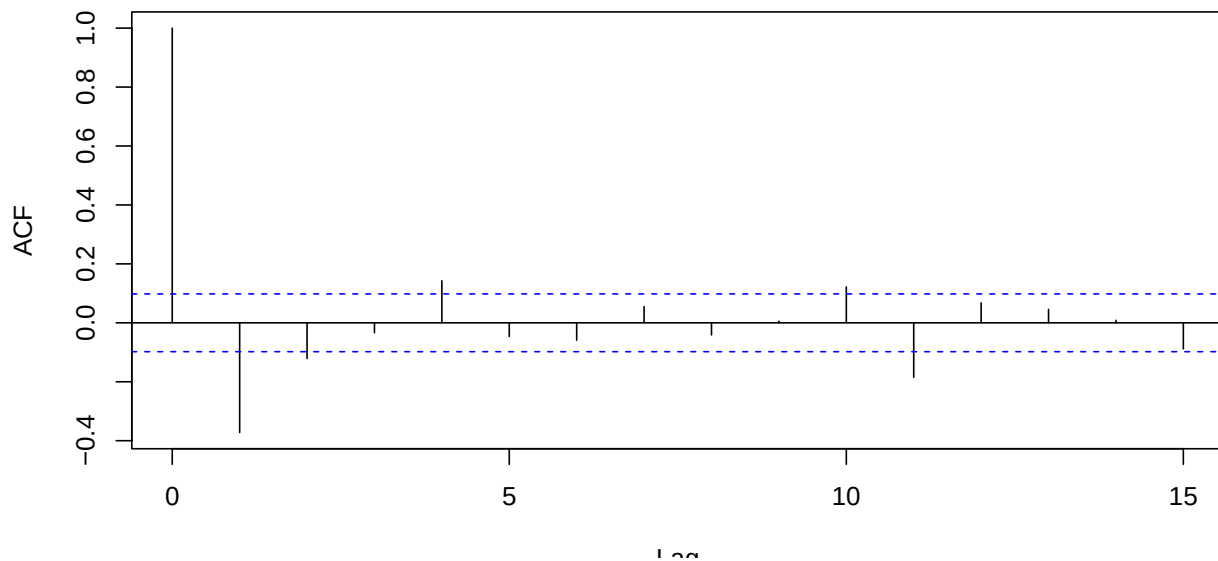
```
)
abline(h = 10, lty = 2, col = "grey45")

acf(
  z_ma1,
  lag.max = 15,
  main = "Exercise 1: MA(1) sample ACF"
)
```

Exercise 1: simulated MA(1) series around mean 10



Exercise 1: MA(1) sample ACF



```
par(mfrow = c(1, 1))
```

How to read the ACF plot:

- The bar at lag 0 is always 1 and is not used to identify the model.
- The key bar is lag 1. It should be negative because the MA coefficient is negative.

- Lags 2 and higher should not show a sustained pattern. Their small bars come from finite-sample noise.

Calculate the reference values in R

For an MA(1), the code version of the lecture's lag-1 formula is $\rho_1 = \theta/(1 + \theta^2)$. We calculate it here so students can compare one theoretical number to the sample ACF bar.

```
theta_1 <- -0.6

# gamma_0 is the theoretical variance. sigma^2 is 1 here, but keep sigma in
# the expression so the code visibly follows the model assumptions.
gamma0_ma1 <- (1 + theta_1^2) * sigma^2

# Theoretical lag-1 autocorrelation for an MA(1).
rho1_ma1 <- theta_1 / (1 + theta_1^2)

data.frame(
  model_mean = 10,
  gamma0 = gamma0_ma1,
  rho1 = rho1_ma1
)

  model_mean gamma0      rho1
1         10   1.36 -0.4411765

# acf(..., plot = FALSE) returns estimated correlations as numbers instead of
# immediately drawing a graph. The first item is lag 0, so item 2 is lag 1.
sample_rho1_ma1 <- acf(z_ma1, lag.max = 1, plot = FALSE)$acf[2]
sample_rho1_ma1

[1] -0.3723774
```

`plot = FALSE` is useful when we need a number rather than a figure. The expression `$acf[2]` has two parts:

- `$acf` retrieves the vector of estimated ACF values stored in the returned object;
- `[2]` selects the second value, because R stores lag 0 first and lag 1 second.

The sample value will be close to, not necessarily equal to, `rho1_ma1`.

For the displayed values in this classroom run:

- `model_mean = 10` is the model's theoretical center. It comes from the constant added after simulation.
- `gamma0 = 1.36` is the theoretical variance γ_0 , using $\sigma = 1$ and $\theta = -0.6$. It describes spread, not the mean and not an ACF.
- `rho1 = -0.4411765` is the theoretical lag-1 autocorrelation. Its negative sign comes from the negative MA coefficient.
- `sample_rho1_ma1` is the estimated lag-1 ACF from the single simulated sequence. A value such as `-0.374728` is reasonably close to `-0.4411765`, but it is an estimate rather than the exact theoretical value.

In the ACF image, the tall bar at lag 0 is always exactly 1 because every series is perfectly correlated with itself. Start model interpretation at lag 1. Here the lag-1 bar is clearly negative and crosses the lower blue reference band. The later bars should not form a stable pattern; an occasional later bar that crosses a band can arise from looking at a finite random sample, and does not turn the simulated model into MA(4) or MA(10).

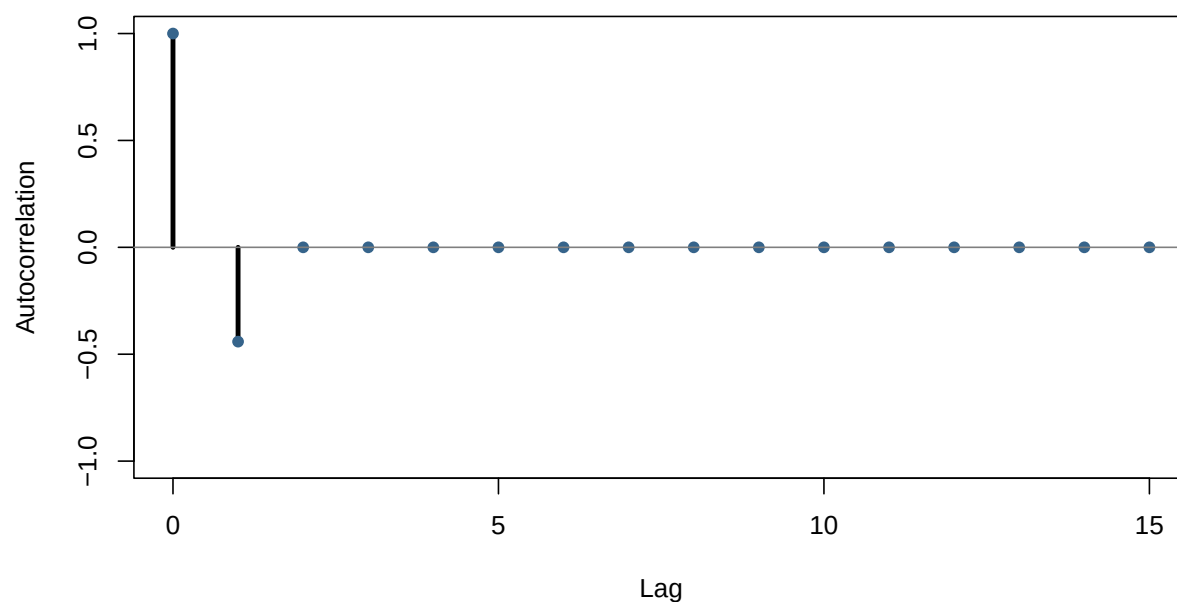
Optional visual check: theoretical versus sample ACF

*# For an MA(1), the theoretical ACF has rho_0 = 1, rho_1 as calculated above,
and rho_k = 0 for all lags k >= 2.*

```
lags <- 0:15
theoretical_ma1_acf <- c(1, rho1_ma1, rep(0, length(lags) - 2))

plot(
  lags,
  theoretical_ma1_acf,
  type = "h",
  lwd = 3,
  ylim = c(-1, 1),
  xlab = "Lag",
  ylab = "Autocorrelation",
  main = "Exercise 1: theoretical MA(1) ACF"
)
points(lags, theoretical_ma1_acf, pch = 16, col = "steelblue4")
abline(h = 0, col = "grey50")
```

Exercise 1: theoretical MA(1) ACF



The theoretical graph has a single negative spike at lag 1. This shape, not an individual finite-sample bar height, is the MA(1) signature.

2. Exercise 2: AR(1)

The exercise model is

$$Y_t = 0.8Y_{t-1} + w_t.$$

The important code change is `list(ar = 0.8)` instead of `list(ma = ...)`. The `ar` argument tells `arima.sim()` that each observation depends on the previous observation. This persistence produces an ACF that decays across many lags.

```
alpha_2 <- 0.8

# Store the innovations separately for the same reason as Exercise 1: this
# vector represents w_t, the fresh white-noise shock at every time point.
innov_ar1 <- rnorm(n = n, mean = 0, sd = sigma)

y_ar1 <- arima.sim(
  model = list(ar = alpha_2),
  n = n,
  innov = innov_ar1
)

# The theoretical variance is gamma_0 = sigma^2 / (1 - alpha^2).
gamma0_ar1 <- sigma^2 / (1 - alpha_2^2)

# For an AR(1), theoretical rho_k = alpha^k.
theoretical_rho_ar1 <- alpha_2^(1:3)

data.frame(
  gamma0 = gamma0_ar1,
  rho1 = theoretical_rho_ar1[1],
  rho2 = theoretical_rho_ar1[2],
  rho3 = theoretical_rho_ar1[3]
)
```

```
      gamma0 rho1 rho2 rho3
1 2.777778  0.8 0.64 0.512
```

The code `alpha_2^(1:3)` has two distinct pieces:

- `1:3` creates the integer vector `c(1, 2, 3)`.
- `^` is exponentiation, and R applies it element by element. Therefore `0.8^(1:3)` returns `c(0.8, 0.64, 0.512)`.

The resulting table has one row because it summarizes one model. Its columns are interpreted as follows:

- `gamma0 = 2.777778` is the model's theoretical variance when `sigma = 1` and `alpha_2 = 0.8`. It is not an ACF value.
- `rho1 = 0.8`, `rho2 = 0.64`, and `rho3 = 0.512` are the theoretical correlations at lags 1, 2, and 3.

This is a compact R way to calculate ρ_1 , ρ_2 , and ρ_3 together rather than writing three nearly identical lines of code.

Compare the ACF and PACF

```
# Three vertical panels need larger top and bottom margins so titles and axis
# labels do not collide when the notebook is rendered.
par(mfrow = c(3, 1), mar = c(4.2, 4.5, 3.2, 1))

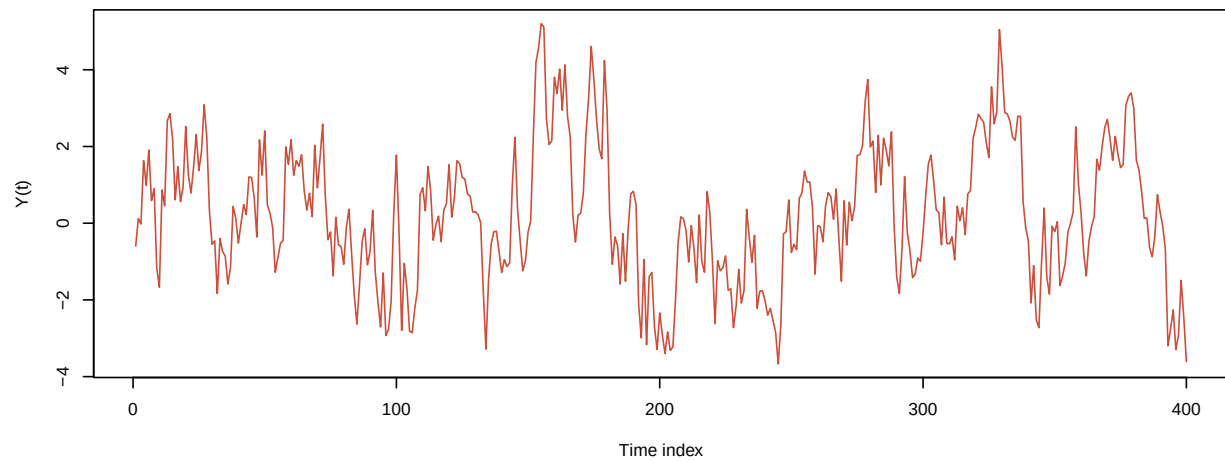
plot(
  y_ar1,
  type = "l",
  col = "tomato3",
```

```
xlab = "Time index",
ylab = "Y(t)",
main = "Exercise 2: simulated AR(1) series"
)

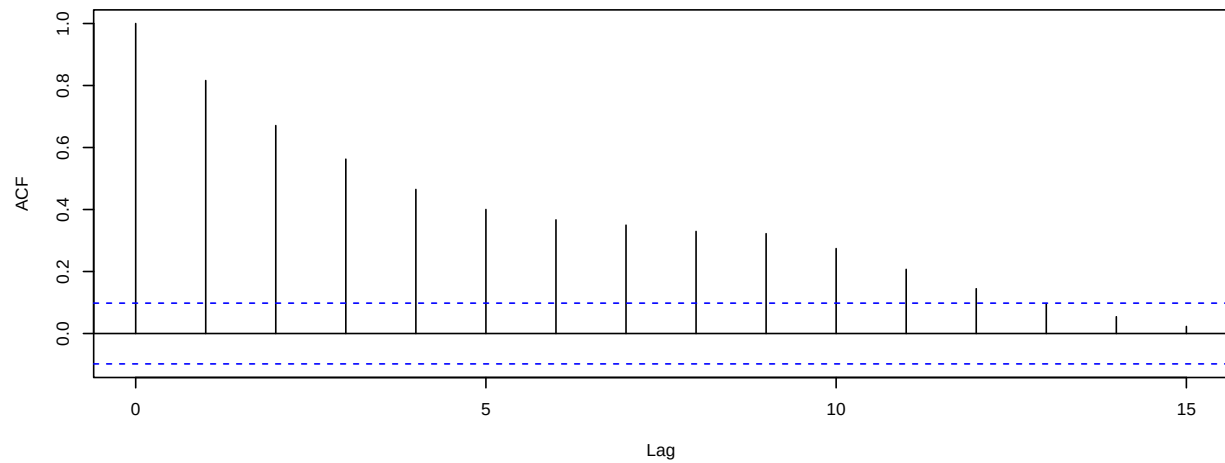
acf(
  y_ar1,
  lag.max = 15,
  main = "Exercise 2: AR(1) sample ACF"
)

pacf(
  y_ar1,
  lag.max = 15,
  main = "Exercise 2: AR(1) sample PACF"
)
```

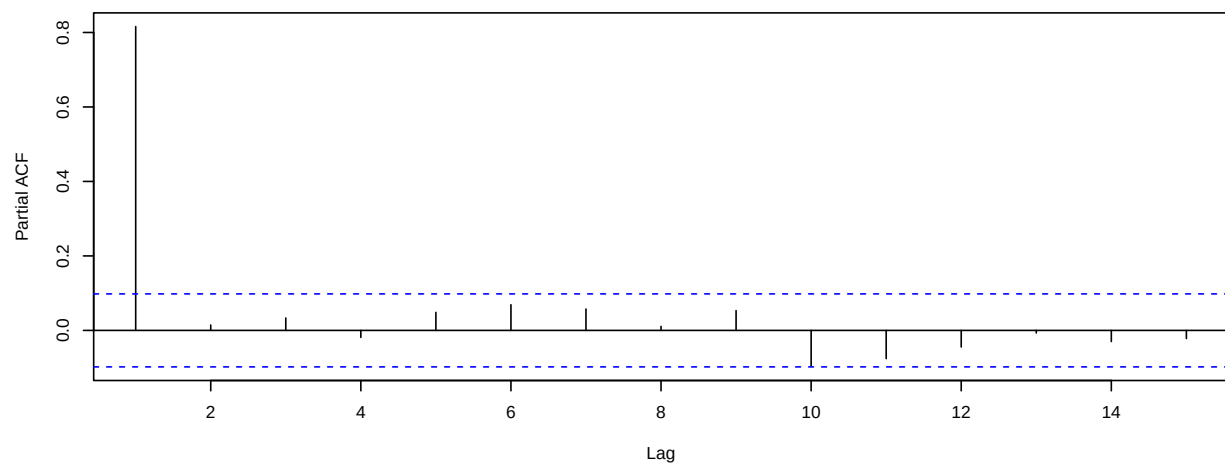
Exercise 2: simulated AR(1) series



Exercise 2: AR(1) sample ACF



Exercise 2: AR(1) sample PACF



```
par(mfrow = c(1, 1), mar = c(5.1, 4.1, 4.1, 2.1))
```

Read the two lower plots together:

- The tall lag-0 ACF bar again equals 1 by definition and should be ignored for order identification.

- The AR(1) **ACF** has a large positive lag-1 bar, then positive bars that decrease gradually. With $\alpha_2 = 0.8$, the theoretical reference values begin at 0.8, 0.64, and 0.512. The exact sample bars will differ because this is one random realization.
- The AR(1) **PACF** has one large lag-1 bar. After accounting for the lag-1 relationship, later lags do not add systematic direct information. Therefore, PACF is the more useful plot for recognizing that the process has AR order 1.
- The blue dashed lines are approximate reference bands. Use them to judge which bars stand out from sampling noise, but make the final decision from the full ACF/PACF pattern rather than from one isolated bar.

Optional numerical comparison of theory and sample values

```
# Retrieve sample ACF estimates at lags 1, 2, and 3. We request lag.max = 3,  
# then skip the stored lag-0 value with [-1].
```

```
sample_rhos_ar1 <- acf(y_ar1, lag.max = 3, plot = FALSE)$acf[-1]
```

```
data.frame(  
  lag = 1:3,  
  theoretical_rho = alpha_2^(1:3),  
  sample_rho = as.numeric(sample_rhos_ar1)  
)
```

	lag	theoretical_rho	sample_rho
1	1	0.800	0.8160436
2	2	0.640	0.6708063
3	3	0.512	0.5624764

`[-1]` means “take all elements except the first.” We remove lag 0 because its correlation is always 1 and would not correspond to the requested lags 1 through 3.

3. Exercise 3: deterministic linear trend plus stationary noise

The exercise model is

$$Y_t = 2 + 0.5t + X_t, \quad X_t = \epsilon_t + 0.5\epsilon_{t-1}.$$

We first simulate X_t as an MA(1). Then we create a time index t and add the deterministic expression $2 + 0.5 * t$. In R, multiplication and addition act element by element, so this adds a different trend value to each observation.

```
# A short sequence is easier to see in a time plot.
```

```
n_trend <- 160
```

```
t_index <- seq_len(n_trend)
```

```
# Simulate the stationary noise term X_t = epsilon_t + 0.5 epsilon_(t-1).  
# Keep its innovations visible just as in Exercises 1 and 2. This vector is  
# the epsilon_t sequence for this particular MA(1) noise component.
```

```
innov_x_ma1 <- rnorm(n = n_trend, mean = 0, sd = sigma)
```

```
x_ma1 <- arima.sim(  
  model = list(ma = 0.5),  
  n = n_trend,  
  innov = innov_x_ma1  
)
```

```

# Each element of trend_part equals 2 + 0.5 times its time index.
trend_part <- 2 + 0.5 * t_index

# Add the same-position elements of trend_part and x_ma1.
y_trend <- trend_part + x_ma1

# Show what the first six time points contain.
head(data.frame(
  time = t_index,
  trend_part = trend_part,
  stationary_noise = as.numeric(x_ma1),
  y = as.numeric(y_trend)
))

```

	time	trend_part	stationary_noise	y
1	1	2.5	-1.80251938	0.6974806
2	2	3.0	-0.29805012	2.7019499
3	3	3.5	0.14781806	3.6478181
4	4	4.0	-0.50770456	3.4922954
5	5	4.5	-0.09967844	4.4003216
6	6	5.0	-0.05489960	4.9451004

The `data.frame()` call creates a small readable table. Each column has a different role:

- `time` is the observation number t ;
- `trend_part` is the non-random expression $2 + 0.5t$;
- `stationary_noise` is the simulated MA(1) term X_t ; and
- `y` is their sum, the series being studied.

Remove the trend with `diff()`

`diff(x)` calculates consecutive differences: second value minus first, third minus second, and so on. It returns one fewer value because the first observation has no earlier observation available for subtraction.

```

# First differences: diff_y_trend[i] = y_trend[i + 1] - y_trend[i].
diff_y_trend <- diff(y_trend)

length(y_trend)

```

```
[1] 160
```

```
length(diff_y_trend)
```

```
[1] 159
```

```

# The model says E(diff(Y_t)) = 0.5. The sample average should be near 0.5,
# but will not be exactly 0.5 because X_t is random.
mean(diff_y_trend)

```

```
[1] 0.50178
```

ACF/PACF before and after differencing

For this particular exercise, differencing has an additional useful consequence. Since

$$X_t = \epsilon_t + 0.5\epsilon_{t-1},$$

the random part of the difference is

$$X_t - X_{t-1} = \epsilon_t - 0.5\epsilon_{t-1} - 0.5\epsilon_{t-2}.$$

So ∇Y_t is a stationary MA(2)-type process around mean 0.5. This predicts a **cutoff in its ACF after lag 2**. We use this as a pattern to look for, while remembering that a finite simulated ACF is noisy.

Six panels let us compare the raw nonstationary series with the differenced series. Wider margins avoid overlap of plot titles and axis labels.

```
par(mfrow = c(2, 3), mar = c(4.3, 4.5, 3.2, 1))

plot(
  y_trend,
  type = "l",
  col = "grey30",
  xlab = "Time index",
  ylab = "Y(t)",
  main = "Exercise 3: trend plus stationary MA(1) noise"
)

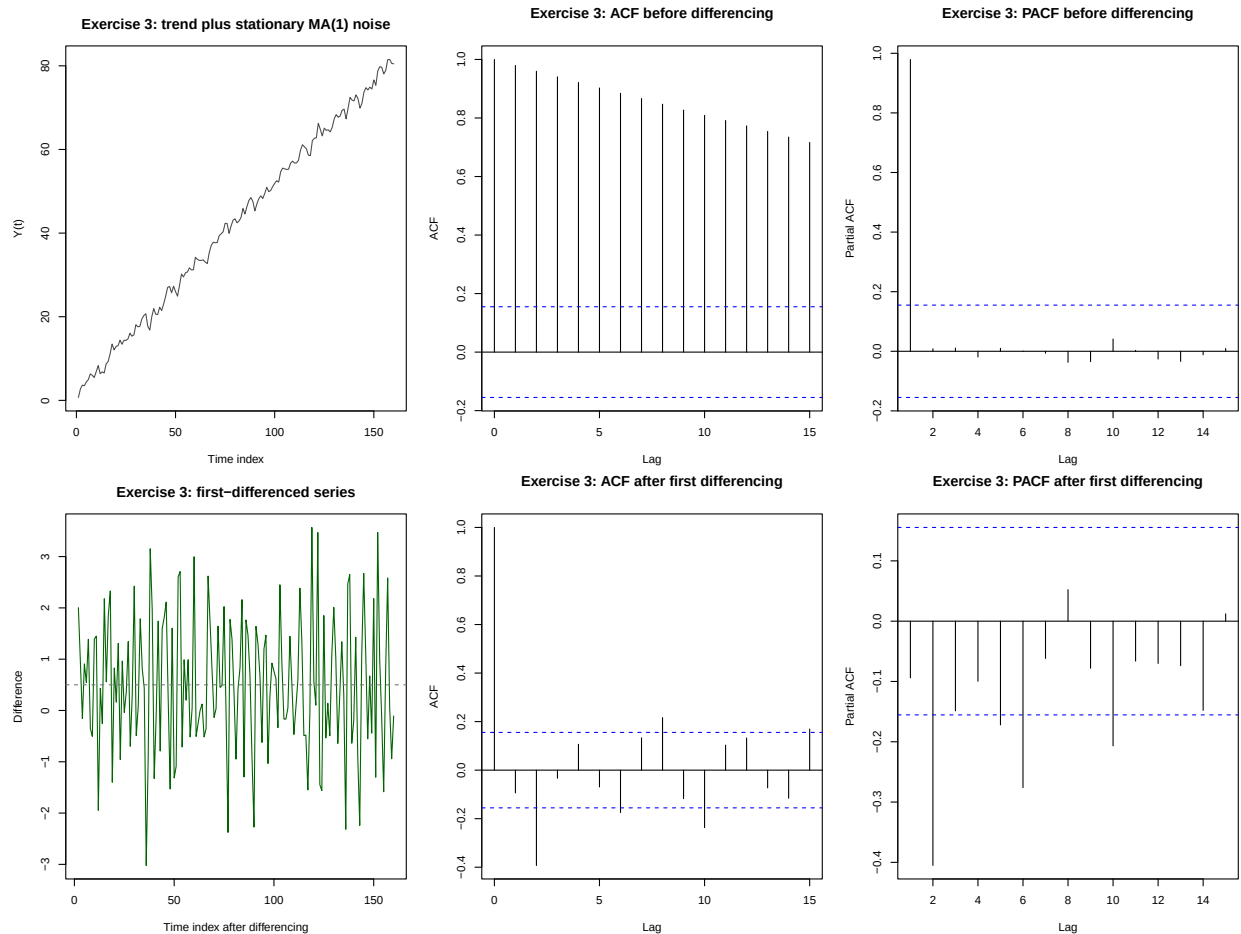
acf(
  y_trend,
  lag.max = 15,
  main = "Exercise 3: ACF before differencing"
)

pacf(
  y_trend,
  lag.max = 15,
  main = "Exercise 3: PACF before differencing"
)

plot(
  diff_y_trend,
  type = "l",
  col = "darkgreen",
  xlab = "Time index after differencing",
  ylab = "Difference",
  main = "Exercise 3: first-differenced series"
)
abline(h = 0.5, lty = 2, col = "grey45")

acf(
  diff_y_trend,
  lag.max = 15,
  main = "Exercise 3: ACF after first differencing"
)

pacf(
  diff_y_trend,
  lag.max = 15,
  main = "Exercise 3: PACF after first differencing"
)
```



```
par(mfrow = c(1, 1), mar = c(5.1, 4.1, 4.1, 2.1))
```

Read the six panels in pairs, from left to right:

1. **Top left, original time plot:** the line rises because `trend_part` rises by 0.5 every period. It does not fluctuate around one fixed level.
2. **Top middle, original ACF:** the bars usually begin very high and decay slowly. A trend makes distant points look similar, so this picture can resemble a persistent AR process even though the data-generating model is not a stationary AR model.
3. **Top right, original PACF:** do not use these PACF bars to declare an AR order. PACF order-identification rules assume a stationary series; the deterministic trend violates that requirement.
4. **Bottom left, differenced time plot:** the long upward drift disappears. The dashed line at 0.5 is the theoretical mean difference, and the remaining fluctuations come from the MA noise.
5. **Bottom middle, differenced ACF:** the first one or two lags may stand out, and then the bars should lose their systematic pattern. That agrees with the MA(2)-type expression above.
6. **Bottom right, differenced PACF:** it normally tails off rather than cleanly cutting off. This complements the ACF pattern: for an MA-type process, use the ACF cutoff to recognize its order, not the PACF.

The key lesson is not to memorize the exact sample bar heights. It is to **difference before using ACF/PACF for AR/MA identification** when the time plot has a clear trend.

4. Exercise 4: random walk with drift

The exercise model is

$$Y_t = Y_{t-1} + 0.3 + w_t.$$

The function `cumsum()` is the most direct R representation of this recursion. It forms cumulative sums: each output value contains all earlier increments. Here each increment is $0.3 + w_t$.

```
n_walk <- 300
drift <- 0.3

# Each innovation has mean 0. Adding drift makes each increment have mean 0.3.
walk_increments <- drift + rnorm(n = n_walk, mean = 0, sd = sigma)

# cumsum() creates Y_t from the successive increments. We use a starting value
# of zero implicitly, so the first displayed value is the first increment.
y_random_walk <- cumsum(walk_increments)

# Taking a first difference recovers the increments, except for the first
# increment because diff() has no preceding displayed value for it.
diff_y_random_walk <- diff(y_random_walk)

head(data.frame(
  increment = walk_increments,
  random_walk = y_random_walk
))
```

```
      increment random_walk
1 -0.2285946 -0.22859464
2 -0.6080402 -0.83663480
3  1.6198991  0.78326430
4 -0.3087706  0.47449375
5 -0.4478177  0.02667606
6  0.5110433  0.53771940
```

```
mean(diff_y_random_walk)
```

```
[1] 0.3178369
```

The mean of `diff_y_random_walk` should be near `drift`, which is 0.3. It is not required to equal 0.3 in this one finite simulation.

```
# Use six panels to compare both ACF and PACF before/after differencing.
# The larger figure and margins keep all titles and axes readable.
par(mfrow = c(2, 3), mar = c(4.3, 4.5, 3.2, 1))
```

```
plot(
  y_random_walk,
  type = "l",
  col = "firebrick3",
  xlab = "Time index",
  ylab = "Y(t)",
  main = "Exercise 4: random walk with drift"
)
```

```
acf(
  y_random_walk,
  lag.max = 15,
  main = "Exercise 4: ACF before differencing"
```

```

)

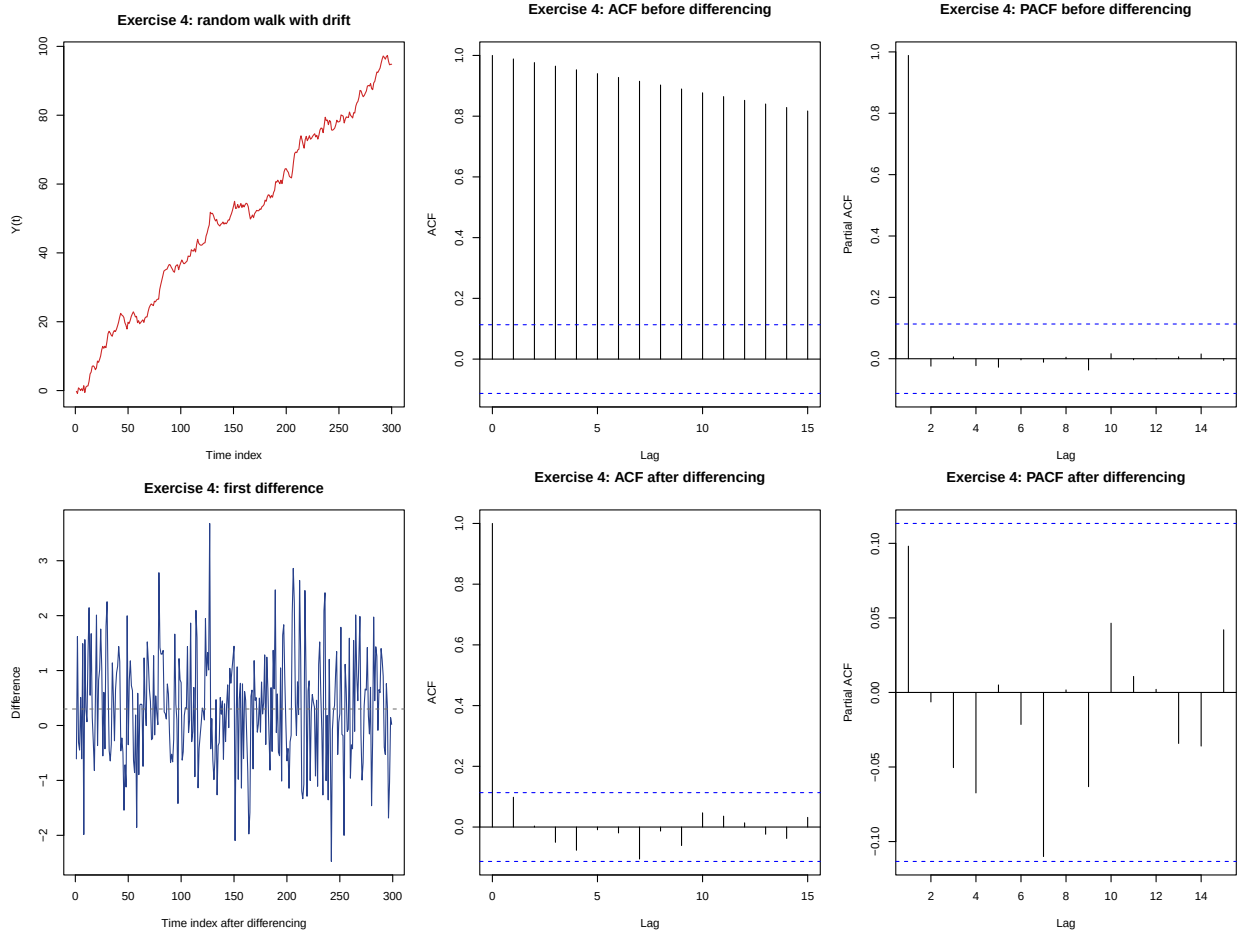
pacf(
  y_random_walk,
  lag.max = 15,
  main = "Exercise 4: PACF before differencing"
)

plot(
  diff_y_random_walk,
  type = "l",
  col = "royalblue4",
  xlab = "Time index after differencing",
  ylab = "Difference",
  main = "Exercise 4: first difference"
)
abline(h = drift, lty = 2, col = "grey45")

acf(
  diff_y_random_walk,
  lag.max = 15,
  main = "Exercise 4: ACF after differencing"
)

pacf(
  diff_y_random_walk,
  lag.max = 15,
  main = "Exercise 4: PACF after differencing"
)

```



```
par(mfrow = c(1, 1), mar = c(5.1, 4.1, 4.1, 2.1))
```

How to explain these six panels in a report:

1. **Top left, random walk:** each value contains the accumulated history of its earlier shocks. A local decline is possible, but the positive `drift = 0.3` makes an overall upward movement common in this simulation. The increasingly remote values from the start should not be described as a stable horizontal process.
2. **Top middle, ACF before differencing:** lag 0 is 1. The other bars remain very high and decline slowly. Adjacent and moderately distant values look similar because they share much of the same accumulated shock history.
3. **Top right, PACF before differencing:** it may show a very large lag-1 bar and other visible bars. Do not call this an AR(1) from the PACF. A random walk is nonstationary, so AR order-identification rules for stationary series do not apply to this raw plot.
4. **Bottom left, first difference:** `diff()` subtracts adjacent cumulative values, so it removes the old accumulated history and leaves the new increment. The grey dashed line at 0.3 marks the theoretical mean increment. The series should jump around this line rather than keep trending upward.
5. **Bottom middle, ACF after differencing:** aside from lag 0, most bars should lie near zero and within the blue reference bands. This is graphical evidence that the differenced series behaves like white noise around its nonzero mean of 0.3.
6. **Bottom right, PACF after differencing:** aside from chance sample bars, it should also have no sustained pattern. ACF and PACF together support the same conclusion: after differencing, there is no AR or MA structure left to identify in this simulated example.

One implementation detail is useful here: `acf()` and `pacf()` remove the sample mean by default. That is

why they can assess serial dependence in `diff_y_random_walk` even though the differenced values fluctuate around 0.3 rather than around zero.

This process is called a **random walk with drift**. The drift is 0.3.

5. Exercise 5: compare MA(1) and AR(1) lag-1 autocorrelation

This exercise is primarily a numerical comparison. We do not need to simulate a series before answering it.

- For an MA(1), $\theta / (1 + \theta^2)$ calculates its theoretical lag-1 autocorrelation.
- For an AR(1), the theoretical lag-1 autocorrelation equals α .

```
theta_5 <- 0.9
alpha_5 <- 0.5

ma1_rho1 <- theta_5 / (1 + theta_5^2)
ar1_rho1 <- alpha_5

comparison <- data.frame(
  process = c("MA(1), theta = 0.9", "AR(1), alpha = 0.5"),
  theoretical_lag_1_acf = c(ma1_rho1, ar1_rho1)
)

comparison

      process theoretical_lag_1_acf
1 MA(1), theta = 0.9           0.4972376
2 AR(1), alpha = 0.5           0.5000000
# which.max() returns the position of the largest value in a numeric vector.
comparison[which.max(comparison$theoretical_lag_1_acf), ]

      process theoretical_lag_1_acf
2 AR(1), alpha = 0.5           0.5
```

Read the four assignment/calculation lines one at a time:

- `theta_5 <- 0.9` stores the MA(1) coefficient from the question in an object called `theta_5`. The arrow `<-` assigns the value on the right to the name on the left. It does not mean “approximately equals.”
- `alpha_5 <- 0.5` stores the AR(1) coefficient in a separately named object. The two parameter names make later formulas readable and prevent accidentally using the MA coefficient in the AR formula.
- `ma1_rho1 <- theta_5 / (1 + theta_5^2)` calculates the MA(1) theoretical lag-1 ACF. R first squares `theta_5`, adds 1, and divides 0.9 by that denominator. Numerically, this is $0.9/(1 + 0.9^2) \approx 0.4972$.
- `ar1_rho1 <- alpha_5` does not need an extra formula because the AR(1) theoretical lag-1 ACF equals its AR coefficient. Thus `ar1_rho1` is exactly 0.5.

The two numbers are deliberately close: the AR(1) has the slightly larger lag-1 autocorrelation. The expression `comparison$theoretical_lag_1_acf` retrieves the named numeric column from the table. `which.max()` then tells R which row has the largest value; the final `[row, ]` selects the full winning row. This provides a transparent code check rather than relying on visual guessing.

Why an MA(1) cannot reach 0.6

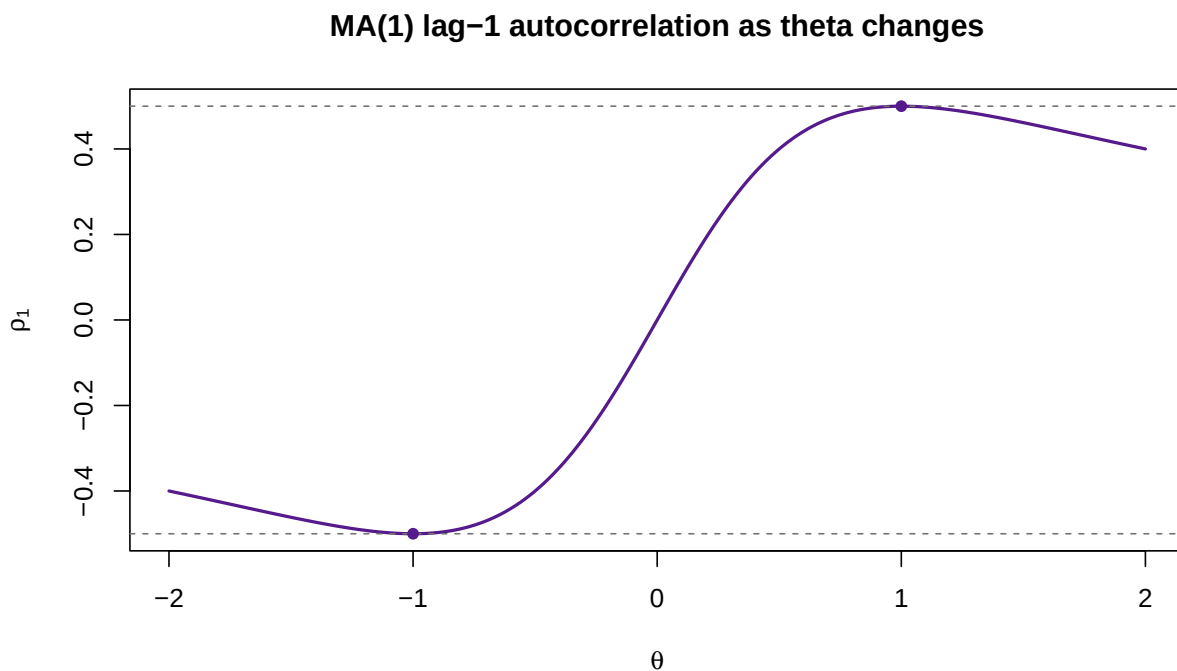
The following plot evaluates the MA(1) formula over many possible values of θ . It shows that the largest positive value occurs at $\theta = 1$ and equals 0.5.

```

# seq(from, to, by) creates a regularly spaced sequence of theta values.
theta_grid <- seq(from = -2, to = 2, by = 0.01)
rho1_grid <- theta_grid / (1 + theta_grid^2)

plot(
  theta_grid,
  rho1_grid,
  type = "l",
  lwd = 2,
  col = "purple4",
  xlab = expression(theta),
  ylab = expression(rho[1]),
  main = "MA(1) lag-1 autocorrelation as theta changes"
)
abline(h = c(-0.5, 0.5), lty = 2, col = "grey45")
points(c(-1, 1), c(-0.5, 0.5), pch = 16, col = "purple4")
text(1, 0.57, "maximum = 0.5", pos = 4)
text(-1, -0.57, "minimum = -0.5", pos = 2)

```



The graph gives the visual reason that $\rho_1 = 0.6$ is impossible for an MA(1). An AR(1) behaves differently: its lag-1 autocorrelation is simply its coefficient α , so an AR(1) can have a lag-1 autocorrelation larger than 0.5 as long as $|\alpha| < 1$.

6. A compact R checklist before submission

Before turning in your code and report, confirm that you can explain the following lines in your own words.

```

# 1. Fix a simulation so another reader can reproduce it.
set.seed(6)

```

```

# 2. Simulate a stationary MA(1) and inspect its ACF.
ma_series <- arima.sim(n = 400, model = list(ma = -0.6))
acf(ma_series, lag.max = 15)

# 3. Simulate an AR(1) and compare ACF and PACF.
ar_series <- arima.sim(n = 400, model = list(ar = 0.8))
acf(ar_series, lag.max = 15)
pacf(ar_series, lag.max = 15)

# 4. Difference a trend or a random walk before interpreting its ACF.
stationary_candidate <- diff(nonstationary_series)
acf(stationary_candidate, lag.max = 15)

```

Your written answers should address these five conclusions:

1. An MA(1) has a lag-1 ACF spike and then cuts off in theory.
2. An AR(1) has an ACF that decays and a PACF that cuts off after lag 1 in theory.
3. A deterministic trend makes the original series nonstationary; `diff()` removes that linear trend.
4. A random walk with drift becomes white noise around its drift after first differencing.
5. A finite sample ACF/PACF plot is an estimate, so it will not match the theoretical shape perfectly.